

Mapa wód PZW — przegląd bezpieczeństwa i audyt dokumentacji

22 września 2026 · dokument wewnętrzny (nie trafia do repozytorium)

Co zostało wdrożone (paczka zmiany-2026-09-22.zip)

Bezpieczeństwo — kod i baza. Funkcja `is_operator()` sprawdza teraz konto po jego identyfikatorze w `auth.users`, wymaga potwierdzonego e-maila, odrzuca konta zablokowane i usunięte oraz ignoruje wielkość liter — uprawnienie wynika z bieżącego stanu konta, nie z treści tokenu; blokada konta w panelu Supabase odbiera zapis natychmiast. Baza dostała ograniczenia CHECK (długości pól, znaki sterujące, obrys współrzędnych, kształt geometrii) i tabelę `historia_zmian` z triggerami (kto, kiedy, stan przed i po każdej zmianie — niedostępna z API), rola `anon` ma odebrane prawa zapisu niezależnie od RLS. Walidacja snapshotu i eksport normalizują typy i pilnują tych samych limitów (parametry w `config.json` → `snapshot.limits`), więc błędny rekord w bazie nie wyrzuci mapy ani nie rozdmucha repozytorium. `esc()` escapuje też apostrof, panel jest odporny na osadzenie w ramce z atrybutem `sandbox`, lista kandydatów nie sięga do prototypu obiektu, a konto spoza allow-listy dostaje po zalogowaniu czytelny komunikat. Workflow publikacji uruchamia się także po nocnym snapshotcie (commit tokenem `GITHUB_TOKEN` nie wyzwała zdarzenia push — bez tego kopia awaryjna na stronie nigdy by się nie odświeżyła); `checkout` nie zostawia poświadczeń; `setup-node` podbity do v7.

Dokumentacja. README skrócony i zsynchronizowany z kodem (usunięte nieścisłości: stan migracji, komendy, opis snapshotu, nawigacja, zgłoszenia). Nowa, nietechniczna `docs/instrukcja-operatora.md` (podlinkowana z ekranu logowania). Z repozytorium wychodzą: specyfikacje projektowe z czerwca (opisują nieistniejące już rozwiązania — CDN, `config.js`, brak CI), raport zbiorników i raport rzek (pliki pośrednie, nieaktualne) oraz wykonane migracje. Zasada: `db/migrate-*.sql` = jeszcze nieuruchomione; po uruchomieniu plik znika z repozytorium (historia w git). Wiki GitHub: **nie** — osobne repozytorium bez PR i CI dryfuje względem kodu, a operator nie szuka wiki; jedyna dokumentacja to README, dwa README narzędzi i instrukcja operatora.

Nie wdrożono (decyzja właściciela): `sessionStorage` dla sesji operatora (sesja kończyłaby się z kartą; wygoda vs współdzielone komputery), przepisanie historii gita, MFA/captcha, ruleset na `main`, para `upload-pages-artifact v5` + `deploy-pages v5`, wyłączenie GraphQL.

Do zrobienia po Twojej stronie

- Rozpakować `zmiany-2026-09-22.zip` do katalogu repozytorium i wykonać komendy z `ZMIANY.md` (przeniesienie workflowów, archiwizacja dokumentów i migracji, commit, push).
- Supabase → SQL Editor: uruchomić `db/migrate-2026-09-22-hardening.sql` (idempotentna; wypisze liczbę operatorów z potwierdzonym kontem).
- Podjąć decyzje z listy powyżej — w szczególności o historii gita (W1).

Załącznik A — pełny raport bezpieczeństwa

Przegląd bezpieczeństwa — mapa wód PZW Okręg Rzeszów

Data: 2026-09-21. Zakres: repozytorium `korntech/mapa-wod-pzw-rzeszow` (klon z pełną historią, HEAD `3c6790f`), strona `https://korntech.github.io/mapa-wod-pzw-rzeszow/`, projekt Supabase `cnmuvkymtacspfrqnhma` (sondy wyłącznie odczytowe, bez zapisu i bez tworzenia kont), workflowy GitHub Actions, zależności npm.

Perspektywa: atakujący zewnętrzny bez konta, użytkownik z kontem Supabase spoza allow-listy, operator (zaufany, ale konto może zostać przejęte), osoba przeglądająca historię publicznego repozytorium.

Podsumowanie

Waga	Liczba
Krytyczne	0
Wysokie	1

Waga	Liczba
Średnie	5
Niskie	9
Informacyjne	12

Fundamenty są poprawne: RLS z allow-listą operatorów działa (potwierdzone sondami na żywo), rejestracja publiczna wyłączona, potwierdzanie e-maili włączone, tabela `operators` i funkcja `is_operator()` niedostępne dla `anon`, wszystkie miejsca renderujące dane z bazy przechodzą przez `esc()`, CSP jest obecna w buildzie produkcyjnym, workflowy przypięte do SHA, `npm audit` czysty. Nie znaleziono XSS ani sekretów w historii.

Największe realne ryzyka nie są w kodzie, lecz w otoczeniu: materiały wewnętrzne wciąż dostępne w historii gita, brak śladu audytowego i kopii przy przejęciu konta operatora, oraz fakt, że bezpieczeństwo allow-listy zależy od dwóch przełączników w panelu Supabase.

Pliki z poprawkami: katalog `patch/` (pełne pliki, struktura jak w repozytorium). Poprawki JS zbudowano (`npm run check`, `npm run build`), migrację SQL przetestowano na lokalnym PostgreSQL 16 z zasymulowanym schematem `auth` i rolami `anon` / `authenticated` (idempotentność, RLS, CHECK, trigger historii).

WYSOKIE

W1. Materiały wewnętrzne (notatki prelegenta, prezentacje robocze, agenda) w historii publicznego repozytorium

Opis. Commit `fae9d4a` („Vite, polskie źródła danych (BDOT10k), automatyzacje”) dodał 22 pliki, które commit `974426c` usunął — ale w publicznym repozytorium nadal są dostępne (`git show fae9d4a:<ścieżka>`), także w interfejsie GitHub pod adresem commitu). Commit `3a49e59` zawiera tylko przeniesienie workflowów (4 pliki, 0 zmian) — nie dodaje nic więcej. Innych commitów z takimi materiałami nie ma.

Dowód (`git log --all --diff-filter=D --name-only`; zawartość zweryfikowana):

Ścieżka w <code>fae9d4a</code>	Treść
<code>agenda-spotkania-pzw.md</code>	agenda negocjacji z Okręgiem (koszty, własność, odpowiedzialność)
<code>docs/architektura/2026-09-21-notatki-prelegenta.{html,pdf}</code>	notatki „tylko do Twojego użytku”, strategia rozmowy z Zarządem („zasada milczącej zgody z terminem 14 dni” itp.)
<code>docs/architektura/2026-09-21-prezentacja-zarzad-robocza.pptx</code>	wersja z notatkami, oznaczona „nie wysyłaj”; slajd 11 zawiera prywatny adres e-mail autora (<code>...@icloud.com</code>), metadane: imię i nazwisko
<code>docs/architektura/2026-09-2{0,1}-prezentacja-zarzad*. {pdf,pptx}</code>	5 wariantów prezentacji dla Zarządu
<code>docs/architektura/2026-09-20-podsumowanie-prac. {html,pdf}</code>	wewnętrzne podsumowanie prac
<code>docs/architektura/2026-09-20-architektura-web-mobile. {html,pdf}</code>	dokument architektury (mniej wrażliwy)
<code>Claude outputs/*</code> (7 plików)	kopie powyższych + zrzut ekranu 3,4 MB (bez danych wrażliwych) + migracja SQL
<code>db/seed.sql</code> , <code>db/seed-rivers-granice.sql</code> , <code>config.js</code> , <code>tools/bdot/*.py</code>	dane i kod — bez sekretów

Scenariusz. Przedstawiciel Okręgu, konkurencyjny wykonawca lub dziennikarz otwiera historię repozytorium i czyta notatki przygotowane do negocjacji; adres prywatny autora trafia do baz spamerskich.

Rekomendacja. Decyzja właściciela — dwie opcje: 1. Przepisanie historii: `git filter-repo --invert-paths --path 'Claude outputs' --path agenda-spotkania-pzw.md --path docs/architektura` (usuwa te ścieżki ze wszystkich commitów), `git push --force --all`, następnie wniosek do GitHub Support o usunięcie osieroconych obiektów z serwera (bez tego `git show fae9d4a:...` przez API GitHub nadal działa), sprawdzenie forków (0 widocznych, ale API było niedostępne z tej sesji) i lokalnych klonów współpracowników. Koszt: zmiana wszystkich hashy po `fae9d4a` (w tym 6 merge’y Dependabota — bez znaczenia). Referencje w README/docs nie wskazują na hashe. 2. Przyjęcie ryzyka (treść jest już ujawniona od kilku godzin; obserwatorów repozytorium najpewniej brak).

Niezależnie od wyboru: dodać do `.gitignore` wzorce `Claude outputs/`, `*.pptx`, `*-notatki-*`, i przed każdym commitem „porządkowym” przeglądać `git status` (commit `fae9d4a` dodał 52 pliki jednym `git add -A`). Poprawka: ustawienia zewnętrzne (GitHub), nie kod.

ŚREDNIE

S1. Brak śladu audytowego i realnej kopii zapasowej przy przejęciu lub pomyłce konta operatora

Opis. Zalogowany operator ma pełne `insert/update/delete` na trzech tabelach przez REST — także masowe (`DELETE /rest/v1/rivers?id=gt.0` z tokenem operatora). Nie ma tabeli historii, kolumny `updated_by` ani kopii poza nocnym snapshotem w gicie. Plan Free Supabase nie ma PITR ani codziennych backupów. Snapshot (`tools/snapshot/export.mjs:64-84`) blokuje eksport tylko przy **spadku** liczby wierszy lub wierzchołków — masowa **podmiana** treści (zasady, nazwy, współrzędne w obrysie) przejdzie i po jednej nocy stanie się „ostatnią dobrą” kopią.

Dowód. `db/schema.sql` (brak tabel audytu), `src/admin.js:418-436` (zapis po `id`), `export.mjs:73-82`.

Scenariusz. Phishing hasła operatora (panel jest publiczny, `admin.html`) → wandal podmienia zasady połowu na 49 zbiornikach o 02:00 → snapshot 03:15 utrwala zmiany w repo → nikt nie wie, kto i co zmienił.

Rekomendacja (kod + procedura): - `patch/db/migrate-2026-09-22-hardening.sql` §5: tabela `historia_zmian` (kto: `auth.uid()` + e-mail, kiedy, stan przed i po, operacja) wypełniana triggerem `after insert/update/delete`, niedostępna z API (RLS bez polityki + `revoke`), czytelna w SQL Editor. Odtworzenie pojedynczego wiersza: `update ... set ... = (przed->>'...')`. - Procedura odtworzenia całości: `git show <ostatni dobry commit>:public/data.json > snap.json && node tools/snapshot/seed-sql.mjs snap.json > seed.sql` i uruchomienie w SQL Editor (uwaga: `truncate ... restart identity` zmienia `id`; migracje w `db/` polegają na `id`, ale są już wykonane). Warto to dopisać do README w sekcji „Baza danych”. - Rozważyć w `export.mjs` próg na liczbę zmienionych rekordów (np. >30% wierszy z inną treścią → wstrzymanie, jak przy spadku liczby wierszy) — nie dołączono, bo wymaga decyzji o progu. - Decyzja właściciela: MFA (TOTP) dla operatorów — Supabase udostępnia MFA w planie Free, wymaga ok. 60 linii w panelu (`enroll + challenge`) i zmiany polityk RLS na `auth.jwt()->>'aal' = 'aal2'`.

S2. Nocny snapshot nigdy nie trafi na stronę — commit wykonany `GITHUB_TOKEN` nie wyzwał `deploy.yml`

Opis. `snapshot.yml` robi `git push` tokenem `GITHUB_TOKEN`. Zdarzenia wywołane tym tokenem (poza `workflow_dispatch / repository_dispatch`) nie uruchamiają innych workflowów — to udokumentowane zachowanie GitHub. `deploy.yml` reaguje tylko na `push` i ręczne uruchomienie, więc `public/data.json` w repozytorium będzie świeższy niż na Pages aż do następnego ręcznego pusha. Fallback „mapa działa ze snapshotu, gdy baza nie odpowiada” (README, prezentacja) będzie w praktyce pokazywał stan sprzed ostatniego pusha kodu.

Dowód. `.github/workflows/snapshot.yml:44-52`, `deploy.yml:6-8`; na żywo `data.json` ma `snapshot: 2026-09-20T20:24:10Z` (tak samo jak w repo — workflow jeszcze nie wykonał commitu; pierwsze uruchomienie planowe 2026-09-22 03:15 UTC). Baza ma 26 zbiorników z lokalizacją przybliżoną, snapshot w repo 27 — zmiana operatora czeka na eksport, a po nim na `deploy`, który bez patcha nie nastąpi.

Rekomendacja. `patch/.github/workflows/deploy.yml`: dodatkowy wyzwalacz `workflow_run` po workflow „Snapshot danych z bazy” (tylko przy `conclusion == 'success'`, `checkout ref: main`). Kosztem jest jeden zbędny build w nocie bez zmian (~1 min minut Actions). Poprawka w kodzie (workflow).

S3. Baza przyjmuje dane, które wywracają mapę publiczną i rozdymają repozytorium

Opis. Tabele nie mają ograniczeń poza `not null` na `n`, `lat`, `lon`. Konto operatora (albo przejęte) może przez REST — z pominięciem formularza panelu — zapisać: `pts` jako obiekt JSON zamiast tablicy, `d/r` jako `null`, `c` spoza `niz/gor`, tekst o długości megabajtów, znaki sterujące. Skutki: - `src/main.js:110-114` — `r.pts` będące obiektem trafia do `L.polyline`, `mid` wychodzi `undefined`, a destrukuryzacja `[lat, lon]` w `navigationUrl` (`main.js:33`) rzuca `TypeError` wewnątrz `forEach` → cała mapa publiczna przestaje się renderować (pusta lista, brak warstw), a nie tylko jeden rekord; - `src/main.js:125,145` — `r.d.slice(0, 70)` przy `null` w snapshotcie (`export.mjs:58-59` nie normalizuje `null`, w przeciwieństwie do `data.js:27-36`) → ten sam efekt po nocnym eksporcie; - `validate.mjs` nie sprawdza typów ani długości pól tekstowych → 50 MB w polu `r` przechodzi walidację, trafia do `public/data.json`, do historii gita i na Pages (limit 1 GB/100 MB na plik).

Dowód. `db/schema.sql:3-33`, `tools/snapshot/validate.mjs:188-214`, `src/main.js:110,125,145`.

Rekomendacja (kod + SQL, dołączono): - `patch/db/migrate-2026-09-22-hardening.sql` §4: CHECK — długości (`n ≤ 200`, `t ≤ 200`, `ha ≤ 40`, `o ≤ 100`, `d/r ≤ 4000`), `c in ('niz','gor')`, `a in (0,1)`, brak znaków sterujących, `lat/lon` w obrysie z

`config.json` (49–51 / 20.5–23.6), `pts` = tablica JSON o 2–20000 elementach. Obecne dane mieszczą się z dużym zapasem (max 429 znaków, 549 wierzchołków). Tekst `<script>...</script>` w nazwie pozostaje dozwolony — to poprawne: ochroną jest `esc()` przy renderowaniu, nie filtr w bazie. - `patch/tools/snapshot/validate.mjs` : te same limity + typy + limit rozmiaru pliku 8 MB + `Number.isFinite` dla współrzędnych (dotąd `"50.1"` `>= 49` przechodziło przez koercję). - `patch/tools/snapshot/export.mjs` i `patch/src/data.js` : normalizacja `null` / złych typów do wartości pustych, `Array.isArray(pts)` .

S4. Allow-lista po e-mailu z tokenu — poprawna dziś, ale zależna od dwóch przełączników w panelu Supabase

Opis. `is_operator()` porównuje `auth.jwt()->'email'` z `operators.email`. Sonda `GET /auth/v1/settings` : `disable_signup: true`, `mailer_autoconfirm: false` — więc obecnie: nikt nie założy konta sam, a zmiana e-maila przez `updateUser({ email })` wymaga kliknięcia linku wysłanego na nowy adres (przy domyślnym „Secure email change” — na oba adresy). Atakujący nie potwierdzi adresu operatora, którego skrzynki nie kontroluje. Ryzyko pojawia się, gdy ktoś w panelu Supabase: - włączy rejestrację i na liście `operators` jest e-mail bez założonego jeszcze konta (np. dopisany „z wyprzedzeniem” dla nowego operatora) → każdy rejestruje się na ten adres i po potwierdzeniu ma zapis; - wyłączy potwierdzanie e-maili (`mailer_autoconfirm`) → `updateUser({ email })` zmienia adres natychmiast, bez potwierdzenia — dowolne konto może przyjąć adres z allow-listy (Supabase odrzuci tylko adres już zajęty przez istniejące konto). Dodatkowo: porównanie jest wrażliwe na wielkość liter (Supabase zapisuje e-maile małymi literami; wpis `Jan.Kowalski@...` na liście nigdy nie zadziała — nie jest to luka, ale przyczyna pomyłek), a funkcja nie sprawdza, czy konto jest potwierdzone/zablokowane.

Dowód. `db/schema.sql:67-77` ; sonda `/auth/v1/settings` (wynik powyżej); `POST /rest/v1/rpc/is_operator` jako anon → 401 42501 (poprawnie); `GET /rest/v1/operators` → 401 42501 (poprawnie).

Rekomendacja (SQL, dołączono). `patch/db/migrate-2026-09-22-hardening.sql` \$1: `is_operator()` łączy `public.operators` z `auth.users` po `auth.uid()` i wymaga `email_confirmed_at is not null`, `deleted_at is null`, `banned_until` w przeszłości, porównuje `lower(email)`; `stable`, `search_path = ''`. Efekt: uprawnienie wynika z bieżącego stanu konta w bazie, nie z treści tokenu sprzed godziny; zablokowanie konta w panelu („Ban user”) natychmiast odbiera zapis. Przetestowano: operator z adresem zapisanym różną wielkością liter → `t`; konto potwierdzone spoza listy → `f`; konto z listy bez potwierdzenia → `f`. Ustawienia zewnętrzne: nie włączać rejestracji ani `autoconfirm`; wpisywać na `operators` dopiero po założeniu konta w Authentication → Users.

S5. Sesja operatora w `localStorage` bez wygaśnięcia

Opis. `supabase-js` zapisuje sesję (`access` + `refresh` token) w `localStorage` pod `sb-<ref>-auth-token`. Refresh token nie wygasa; ograniczenie czasu sesji („Time-box user sessions”, „Inactivity timeout”) jest w planie Pro. Sesja na współdzielonym komputerze (biuro Okręgu, kiosk) trwa do ręcznego „Wyloguj”. `signOut()` w `data.js:82-108` jest poprawny wobec `supabase-js 2.116` (klucz zgodny z `sb.auth.storageKey`, token wysyłany tylko do origin Supabase, `keepalive`), ale nie rozwiązuje problemu porzuconej karty.

Dowód. `src/data.js:15,82-108` ; `node_modules/@supabase/supabase-js/dist/index.mjs:651` .

Rekomendacja (kod, dołączono). `patch/src/data.js` : `createClient(..., { auth: { storage: sessionStorage, detectSessionInUrl: false } })` — sesja kończy się z kartą; `signOut()` czyta ten sam magazyn. `detectSessionInUrl: false` wyłącza przetwarzanie `#access_token=...` z adresu URL na obu stronach (panel używa wyłącznie hasła; linki odzyskiwania hasła i tak nie mają obsługi w UI). Skutek dla operatora: logowanie po każdym otwarciu nowej karty. Jeśli to niepożądane — wycofać ten fragment patcha, pozostałe zmiany w pliku są niezależne.

NISKIE

N1. Frame-busting w `admin.js` da się obejść atrybutem `sandbox` ; `frame-ancestors` niedostępne

Opis. `admin.js:19` ustawia `window.top.location`. Ramka `<iframe sandbox="allow-scripts allow-same-origin allow-forms">` blokuje nawigację okna nadrzędnego (zależnie od przeglądarki: wyjątek `SecurityError` albo tylko wpis w konsoli), a jeśli blokada jest cicha, panel dalej działa w ramce. CSP w `<meta>` ignoruje `frame-ancestors` ; GitHub Pages nie wysyła `X-Frame-Options` (sprawdzone nagłówki na żywo). Wpływ ograniczony: partycjonowanie storage w nowych przeglądarkach sprawia, że ramka z cudzej domeny nie widzi sesji operatora, a usunięcie wymaga `confirm()` .

Rekomendacja (kod, dołączono). `patch/src/admin.js` : w ramce strona jest ukrywana (`documentElement.style.display = 'none'`), próba nawigacji w `try/catch`, a moduł przerywa inicjalizację (`throw`) — panel nie wystartuje niezależnie od tego, czy przekierowanie zadziała.

N2. Zgubione podbicia Dependabota — przypięte SHA nie odpowiadają komentarzom ani zmergowanym PR

Opis. PR #5 (`actions/setup-node 4.4.0 → 7.0.0`) i PR #2 (`upload-pages-artifact 3.0.1 → 5.0.0`) zostały zmergowane, ale merge `b664c14 / d474112` („Merge branch 'main' into dependabot/...”) przywrócił stare wersje. Weryfikacja `git ls-remote --tags :`

Akcja	SHA w plikach	Faktyczny tag	Komentarz w pliku
<code>actions/checkout</code>	<code>3d3c42e...</code>	<code>v7.0.1</code>	<code>v7.0.1 ✓</code>
<code>actions/setup-node</code>	<code>49933ea...</code>	<code>v4.4.0</code>	„v4” (PR #5 → 7.0.0 zgubiony)
<code>actions/configure-pages</code>	<code>45bfe01...</code>	<code>v6.0.0</code>	<code>v6.0.0 ✓</code>
<code>actions/upload-pages-artifact</code>	<code>56afc60...</code>	<code>v3.0.1</code>	„v3” (PR #2 → 5.0.0 zgubiony)
<code>actions/deploy-pages</code>	<code>d6db901...</code>	<code>v4.0.5</code>	„v4”
<code>actions/github-script</code>	<code>3a2844b...</code>	<code>v9.0.0</code>	<code>v9.0.0 ✓</code>

Wszystkie SHA są prawdziwymi tagami upstream (brak podmiany), więc to higiena, nie luka. Dependabot otworzy PR-y ponownie w najbliższym cyklu tygodniowym.

Rekomendacja. Dołączone workflowy pinują `setup-node@8207627...` (v7.0.0, wersja z zaakceptowanego PR #5). `upload-pages-artifact v5` wymaga sparowania z `deploy-pages v5` — pozostawiono do PR Dependabota. Auto-merge Dependabota: **nie jest włączony** (brak workflow z `dependabot/fetch-metadata / gh pr merge`, brak `auto-merge` w konfiguracji) — potwierdzone, tak ma zostać.

N3. `ci.yml` uruchamia kod z pull requestów z forków (skrypty `npm ci`, `vite.config.js`)

Opis. Zdarzenie `pull_request` z forka wykonuje `npm ci` (skrypty lifecycle z `package-lock.json` z PR) i build z `vite.config.js` z PR. Job ma `contents: read`, brak sekretów, token forka jest tylko do odczytu, cache npm jest izolowany per gałąź PR — nie ma czego wykraść. Ryzyko resztkowe: użycie runnera do kopania/skanowania na koszt limitu minut.

Rekomendacja (dołączono). `persist-credentials: false` przy checkout w `ci.yml` i `deploy.yml` (token nie zostaje w `.git/config` na czas `npm ci`). Ustawienie zewnętrzne: Settings → Actions → „Require approval for all outside collaborators” (domyślnie wymagana jest akceptacja tylko dla nowych kontrybutorów — wystarczające).

N4. `esc()` nie escapuje apostrofu

Opis. `data.js:111-116` zamienia `& < > "`. Wszystkie atrybuty w szablonach są w cudzysłowach (`main.js:47-48,178`, `admin.js:192`, `wykaz.js:8`), więc dziś to nie jest luka — ale każdy przyszły `href='...'` stanie się XSS.

Rekomendacja (dołączono). `patch/src/data.js`: dodane `' → '`.

N5. `candidates[o.n]` sięga do prototypu — nazwa „constructor” wywraca panel

Opis. `admin.js:257` — `candidates['constructor']` zwraca `Object`, `.length === 1`, `.map` nie istnieje → `TypeError` w `select()` zanim otworzy się edytor. Zbiornik o nazwie `constructor`, `toString`, `__proto__` itp. staje się nieedytowalny w panelu (dane z `kandydaci-zbiorniki.json` w repo, więc bez wstrzyknięcia; klasa „prototype pollution” bez skutków bezpieczeństwa).

Rekomendacja (dołączono). `Object.hasOwn` + walidacja typów przy wczytaniu kandydatów.

N6. Nadużycie klucza publicznego: nielimitowany odczyt i limit transferu planu Free

Opis. Pełny odczyt trzech tabel to ~180 KB (głównie `rivers.pts`). Plan Free: 5 GB egress/mies. ≈ 28 tys. pełnych odczytów; po przekroczeniu projekt jest ograniczany. Supabase nie ma limitu zapytań REST per IP dla `anon`. Skutek dla użytkowników: mapa przełącza się na snapshot (projektowane zachowanie), panel operatora przestaje działać do końca okresu rozliczeniowego lub przejścia na Pro.

Rekomendacja. Brak poprawki w kodzie o sensownym koszcie. Monitorować Usage w panelu Supabase; `healthcheck.yml` już wykryje 5xx. Opcjonalnie (decyzja): strona publiczna mogłaby czytać tylko snapshot, a bazę odpytywać wyłącznie o `updated_at` (jeden mały request), pobierając pełne tabele tylko przy zmianie — zmniejsza egress ~100×, ale komplikuje kod.

N7. Brak MFA i ochrony logowania po naszej stronie; rate-limit tylko po stronie Supabase

Opis. Logowanie e-mail + hasło bez drugiego składnika i bez captcha. Limity Supabase (dokumentacja): `/auth/v1/token 150` żądań / 5 min / IP (seria do 30), reset hasła 1/60 s/użytkownik, wysyłka e-mail 2/h z wbudowanego SMTP. Ochrona przed hasłami

z wycieków (HIBP) jest w planie Pro. Dla ataku słownikowego z jednego IP to ~43 tys. prób/dobę; z botnetu — bez limitu. Bezpieczeństwo zależy od jakości haseł operatorów.

Rekomendacja (decyzja właściciela; ustawienia zewnętrzne + kod). W panelu Supabase: Authentication → Attack Protection → włączyć Cloudflare Turnstile/hCaptcha (Free) — wymaga przekazania `captchaToken` w `signInWithPassword` (kilka linii w `admin.js` + widget); ustawić minimalną długość hasła ≥ 12 i wymagania znaków (Free). W dalszej kolejności MFA TOTP (patrz S1).

N8. Ekspozycja GraphQL i brak kontroli, czy operator używa panelu

Opis. `POST /graphql/v1` odpowiada 200 (pg_graphql włączony) — te same tabele i RLS, więc nie zwiększa uprawnień, ale poszerza powierzchnię i ułatwia masowe zapytania. OpenAPI `GET /rest/v1/` zwraca pustą definicję (dobrze — schemat nie jest wypisywany).

Rekomendacja. Ustawienie zewnętrzne: Settings → API → wyłączyć GraphQL, jeśli nieużywany.

N9. `admin.html:167` — `target="_blank"` bez `rel="noopener"`

Opis. Link do mapy publicznej (ten sam origin; nowoczesne przeglądarki i tak stosują `noopener` domyślnie dla `_blank`). Bez skutków, niespójność z resztą plików.

Rekomendacja. Dodać `rel="noopener"`; nie dołączono osobnego pliku dla jednej linii HTML.

INFORMACYJNE

I1. Klucz `sb_publishable_...` w repozytorium — zgodnie z projektem, RLS potwierdzone

Klucz publishable jest przeznaczony do przeglądarki; bezpieczeństwo zapewnia RLS. Sondy (anon): `operators` → 401/42501; `rpc/is_operator` → 401/42501; `storage/v1/bucket` → `[]`; `rest/v1/` → pusty OpenAPI; bez `apikey` → 401; `auth/v1/settings`: rejestracja wyłączona, autoconfirm wyłączony, anonimowi wyłączeni, brak OAuth, brak passkeys. Zapis jako anon przetestowano lokalnie na tym samym `schema.sql` (odmowa). Zapisów na żywo nie wykonywano.

I2. XSS — wynik negatywny (wszystkie ujścia sprawdzone)

Miejsce	Dane	Ochrona
<code>main.js:41-49</code> <code>popup(title, tag, body, rules)</code>	baza/snapshot	<code>esc()</code>
<code>main.js:47-48</code> <code>href</code> nawigacji i zgłoszenia	nazwa z bazy	<code>encodeURIComponent(name)</code> w szablonie z <code>config.json</code> + <code>esc()</code> całego URL; schemat (<code>geo: / https: / mailto: </code>) pochodzi z konfiguracji, nie z danych
<code>main.js:54-60</code> <code>reportUrl</code>	nazwa, lat/lon	<code>encodeURIComponent</code> całych <code>subject / body</code>
<code>main.js:140</code> <code>popup</code> granic; <code>main.js:175-184</code> lista	baza	<code>esc()</code> ; <code>data-i</code> = indeks liczbowy; kolory ze stałych
<code>admin.js:184-195</code> lista; 263-272 kandydaci	baza; JSON w repo	<code>esc()</code> ; <code>data-id = bigint</code> ; liczby przez <code>toFixed / Math.round</code>
<code>admin.js:115</code> e-mail, 133,424,437 błędy, toasty	Supabase	<code>textContent</code>
<code>wykaz.js:8-48</code> tabele	baza/snapshot	<code>esc()</code> na każdej komórce
<code>basemaps.js:92,115</code> ; <code>main.js:243</code> ; <code>admin.js:64</code>	<code>config.json</code>	zaufana konfiguracja w repo
<code>L.control.layers</code> tytuły warstw	<code>config.json</code>	j.w.
Snapshot i kandydaci: <code>JSON.parse</code> bez <code>Object.assign / deep-merge</code> — klucz <code>__proto__</code> w JSON tworzy własność		
własną, nie zmienia prototypu. <code>data.json</code> na żywo: 0 tagów HTML, 0 URL, 0 e-maili, 0 telefonów.		

Miejsce	Dane	Ochrona
kandydaci-zbiorniki.json : bez ścieżek lokalnych, e-maili, URL — tylko dane BDOT10k (publiczne).		

13. CSP: style-src 'unsafe-inline' jest dziś konieczne

Wymagają go: bloki `<style>` w trzech plikach HTML oraz atrybuty `style="..."` w szablonach `innerHTML` (`main.js:42,178,180`, `index.html:54-57,75-82`). `basemaps.js:153,161` używa `cssText` przez `CSSOM` — to CSP dopuszcza. Usunięcie wymagałoby przeniesienia stylów do klas i hashy `'sha256-...'` dla `<style>`; zysk niewielki (brak miejsc, gdzie atakujący kontroluje style). `img-src blob`: wydaje się nieużywane (Leaflet i Geoman nie tworzą blobów) — można usunąć przy okazji. `connect-src / img-src` domykają się: kafele Geoportalu tylko jako `<img>`, fetch tylko do `self` (snapshot) i Supabase. `frame-ancestors`, `report-uri` w `<meta>` nie działają — bez alternatywy na GitHub Pages.

14. Strona na żywo — zgodność z buildem

`index.html`, `admin.html`, `wykaz.html`: meta CSP identyczna z `vite.config.js`, skrypty wyłącznie `/mapa-wod-pzw-rzeszow/assets/*.js` (`type="module"` `crossorigin`), brak skryptów zewnętrznych, brak `<link>` do CDN. Nagłówki Pages: `strict-transport-security: max-age=31556952`, `access-control-allow-origin: *` (statyczna treść, bez znaczenia), `cache-control: max-age=600`; brak `X-Frame-Options`, `Content-Security-Policy`, `Referrer-Policy`, `Permissions-Policy` (Pages nie pozwala ich ustawić). `admin.html` jest publicznie dostępny i podlinkowany z nagłówka mapy.

15. Historia gita — sekrety: brak

Przeszukano wszystkie bloy (`git rev-list --all | xargs git grep`) pod kątem `service_role`, `JWT` (`eyJ...`), `sk_`, `sbp_`, `ghp_`, `github_pat_`, kluczy prywatnych, `password / hasło`, e-maili i telefonów. Wyniki: - `9dcd008`, `e7fba35` — `db/seed.mjs` (usunięty w `038fba6`): `SUPABASE_SERVICE_KEY="eyJ...service_role..."` to placeholder w komentarzu, nie klucz; `README` w `e7fba35` opisuje użycie klucza `service_role` lokalnie; - klucz publishable w `config.js / config.json` we wszystkich commitach od `e7fba35` (zgodnie z projektem); - e-mail: `operator@przyklad.pl` (`README`), `snapshot-bot@users.noreply.github.com`; w metadanych commitów `kamil.kornacki@perge20.com` (19 commitów — adres służbowy autora, standardowe w gicie); prywatny ... `@icloud.com` wyłącznie w `pptx` z `W1`; - telefony: brak; klucze prywatne/tokens: brak.

16. signOut() — ocena implementacji

Poprawna wobec `supabase-js 2.116.0`: `sb.auth.storageKey` istnieje i równa się `sb-<ref>-auth-token` (`index.mjs:651`), struktura zapisanej sesji ma `access_token` na najwyższym poziomie, `POST /auth/v1/logout?scope=global` z `apikey + Bearer` to ten sam wywołany przez `admin.signOut`. Token nie wycieka (jedyń odbiorca: origin Supabase, HTTPS). Krucho strona: zależność od wewnętrznego formatu storage — przy zmianie formatu w przyszłej wersji token nie zostanie wysłany (sesja serwerowa pozostanie aktywna do wygaśnięcia refresh tokenu), ale lokalna sesja i tak zniknie. Alternatywa bez tej kruchości: `await Promise.race([sb.auth.signOut(), timeout(1500)])` przed `location.replace`.

17. Snapshot jako źródło wstrzyknięcia do repozytorium — ocena

`export.mjs` zapisuje wyłącznie `JSON.stringify(next)` do pliku o nazwie z `config.json`; `git add / commit` operują na stałej ścieżce; komunikat commita to stała + data. Brak możliwości wykonania kodu przez treść bazy. Treść (HTML/ `<script>` w nazwie) trafia do `data.json` jako string i jest renderowana przez `esc()` (12). Jedyne wektory to objętość i typy (S3). `healthcheck.yml`: `REPORT` budowany tylko z kodów HTTP; `github-script` dostaje go przez `env` — bez wstrzyknięcia.

18. Uprawnienia workflowów i concurrency

`ci.yml`: `contents: read` ✓. `deploy.yml`: `contents: read`, `pages: write`, `id-token: write`, `concurrency: pages` ✓. `snapshot.yml`: `contents: write` (konieczne do pusha), `concurrency` bez anulowania ✓. `healthcheck.yml`: `issues: write`, `contents: read` ✓. Push `snapshot-bot` bezpośrednio do `main` oznacza, że `main` nie ma reguły „wymagaj PR” (nie dało się zweryfikować przez API z tej sesji). Rekomendowany ruleset: blokada `force-push` i usunięcia gałęzi `main` (nie przeszkadza botowi), 2FA na koncie właściciela (wymagane przez GitHub), przegląd listy „Deploy keys”/„Personal access tokens” raz na kwartał.

19. Przejęcie konta GitHub właściciela — model

Konto = kontrola kodu na Pages (podmiana `admin.js` na wersję kradnącą hasła operatorów, zmiana `config.json` na własny projekt Supabase) i sekretu OIDC dla Pages. Środki: 2FA z kluczem sprzętowym lub passkey, brak PAT o szerokim zakresie,

environment `github-pages` z „Required reviewers” (blokuje automatyczny deploy — kompromis do decyzji), włączone „Secret scanning” i „Push protection” (bezpłatne w publicznych repo). Poza zakresem kodu.

I10. Phishing panelu operatora i „ukrywanie” `admin.html`

Panel jest widoczny w kodzie, więc ukrycie linku w nagłówku nic nie daje (adres jest w repo i w `dist/`). Klon panelu na podobnej domenie (`korntech-github.io`) jest realnym wektorem; obroną są: MFA/captcha (N7), menedżer haseł operatora (autouzupełnianie nie zadziała na fałszywej domenie — warto to powiedzieć na szkoleniu), stały adres w zakładce. Alternatywa do decyzji: przeniesienie panelu na osobny adres z własną domeną i `Cloudflare Access` /podobnym — poza planem „0 zł”.

I11. Geoportal (WMTS GUGiK)

Kafle pobiera przeglądarka użytkownika bezpośrednio z `mapy.geoportal.gov.pl` ; strona nie proxyuje ani nie agreguje ruchu, `updateWhenIdle: true` , `keepBuffer: 2` . Ewentualny DoS na Geoportal nie jest wynikiem naszej strony; awaria Geoportalu jest obsługiwana fallbackiem (`basemaps.js:129-146`). Warunki PZGiK: podanie źródła — spełnione.

I12. Zależności

`npm audit` : 0 podatności (prod i dev). Wersje: `@supabase/supabase-js` 2.116.0 (aktualna), `leaflet` 1.9.4 (ostatnie wydanie 2023, projekt aktywny w gałęzi 2.0), `proj4` 2.22.0, `@geoman-io/leaflet-geoman-free` 2.20.2 — licencja MIT (potwierdzona w pakiecie), wydanie z 2026-09-21. `proj4leaflet` 1.0.2 — **ostatnie wydanie 2017-08-14**, repozytorium `kartena/Proj4Leaflet` nieaktywne; ~300 linii kodu bez zależności sieciowych i bez skryptów instalacyjnych, podatności nieznane. Ryzyko: brak poprawek pod Leaflet 2.0; przy migracji trzeba będzie wchłonąć plik do `src/` lub zastąpić własną klasą CRS (`proj4` wystarczy). Żadna zależność nie ma skryptów `preinstall/install/postinstall` (sprawdzone w `node_modules`). Dokumentacyjna drobnostka: README oznacza migracje uprawnień i `kraina-pstraga` jako „do uruchomienia”, a baza na żywo ma już 28 rzek i odmawia dostępu do `operators` — obie są wykonane.

Poprawki do wdrożenia od razu (`patch/`)

Plik	Zmiana	Finding
<code>patch/src/data.js</code>	<code>sessionStorage</code> dla sesji, <code>detectSessionInUrl: false</code> , normalizacja typów z bazy, <code>esc()</code> z apostrofem	S5, S3, N4
<code>patch/src/admin.js</code>	frame-busting odporny na <code>sandbox</code> , <code>Object.hasOwn</code> dla kandydatów, komunikat dla konta spoza <code>allow-listy</code> (<code>rpc('is_operator')</code>)	N1, N5
<code>patch/tools/snapshot/export.mjs</code>	normalizacja <code>null</code> /typów przed zapisem snapshotu	S3
<code>patch/tools/snapshot/validate.mjs</code>	typy i długości pól, znaki sterujące, <code>Number.isFinite</code> , limit 8 MB	S3
<code>patch/db/migrate-2026-09-22-hardening.sql</code>	<code>is_operator()</code> po <code>auth.uid()</code> + potwierdzony e-mail + <code>lower()</code> ; <code>search_path=''</code> ; <code>revoke</code> zapisu dla <code>anon</code> ;CHECK; <code>historia_zmian</code> z triggerami	S4, S3, S1
<code>patch/.github/workflows/deploy.yml</code>	wyzwalacz <code>workflow_run</code> po snapshotcie; <code>persist-credentials: false</code> ; <code>setup-node v7.0.0</code>	S2, N2, N3
<code>patch/.github/workflows/ci.yml</code>	<code>persist-credentials: false</code> ; <code>setup-node v7.0.0</code>	N2, N3
<code>patch/.github/workflows/snapshot.yml</code>	<code>setup-node v7.0.0</code> ; <code>git add --</code>	N2

Kolejność: najpierw migracja SQL (SQL Editor; skrypt wypisze liczbę operatorów z potwierdzonym kontem i przerwie się, gdy żaden nie ma konta), potem commit plików JS/workflowów. Migracja jest idempotentna i nie zmienia danych poza zamianą `null` na wartości puste w polach tekstowych.

Co wymaga decyzji właściciela

- W1 — przepisać historię gita (i zgłosić do GitHub Support), czy przyjąć ryzyko.
- S1/N7 — MFA TOTP i/lub captcha na logowaniu operatora (zmiana kodu panelu + ustawienia Supabase).
- S5 — czy sesja operatora ma kończyć się z kartą (`sessionStorage`; dołączony patch) — wygoda vs. bezpieczeństwo na współdzielonych komputerach.
- N2 — podbić parę `upload-pages-artifact v5` + `deploy-pages v5` (PR Dependabota) czy zostać przy v3/v4.
- I8/I9 — ruleset na `main` (blokada force-push), „Required reviewers” dla environment `github-pages` , `push protection`; wyłączenie GraphQL (N8).

Załącznik B — audyt dokumentacji

Audyt dokumentacji repozytorium `mapa-wod-pzw-rzeszow`

Zakres: wszystkie pliki o charakterze dokumentacji (README, README narzędzi, `docs/`, nagłówki SQL, workflowów i skryptów, `config.json`, `package.json`, `LICENSE`) sprawdzone względem kodu w `src/`, `tools/`, `db/`, `.github/` i danych w `public/`.
Stan repo: commit `3c6790f`.

Czytelnicy: (a) wędkarz, (b) programista-wolontariusz, (c) operator Okręgu (nietechniczny), (d) osoba techniczna przejmująca utrzymanie.

1. Decyzje w skrócie

- README zostaje jedynym dokumentem „wejściowym”**, skrócony i bez stanu („uruchomiona / do uruchomienia”), bez narracji historycznej i bez wewnętrznych szczegółów (region Supabase itp.). Wersja do podmiany: `README.md` obok tego raportu.
- Zostają dwa README narzędzi** (`tools/bdot/README.md`, `tools/qa/README.md`) — są zgodne z kodem i są jedyną dokumentacją tych pakietów. Jedna drobna poprawka w `tools/bdot/README.md` (patrz §3).
- Powstaje jedna nowa strona: `docs/instrukcja-operatora.md`** (dla czytelnika c). To jedyny dokument, którego brakuje: operator nietechniczny nie ma dziś żadnego opisu panelu. Wersja gotowa obok tego raportu. Do panelu warto dodać jeden link (`admin.html`, karta logowania) do tej strony.
- Usuwane z repo (przeniesione do `_prywatne/`, który jest w `.gitignore`): `docs/specs/*`** (4 pliki), `docs/raport-zbiorniki.md`, `docs/rzeki-raport.json`. Uzasadnienie w tabeli §2. Treści operacyjne z raportu (przypadki podejrzane) idą do Issues.
- Wiki GitHub: nie.** Uzasadnienie w §5.
- Katalog `docs/` po sprzątnięciu zawiera wyłącznie `instrukcja-operatora.md`.

2. Ocena plików

Plik	Dla kogo	Aktualny wobec kodu?	Decyzja	Uzasadnienie
<code>README.md</code>	a, b, d	w większości tak; lista nieścisłości w §3	skrócić / przeredagować	dubluje opis snapshotu z narzędziami, zawiera stan migracji (drift), szczegóły wewnętrzne (region Frankfurt), „znane braki” z liczbami, które starzeją się z każdym zapisem w panelu
<code>tools/bdot/README.md</code>	b, d	tak — podkomendy, opcje (<code>--od/--do</code> , <code>--paczki</code> , <code>--wfs-plik</code> , <code>--data-raportu</code> , <code>--data-aktualizacji</code> , <code>--log-level</code>), sekcje <code>config.json</code> , nazwy plików z ścieżki zgodne z <code>cli.py</code> i <code>config.json</code>	zostaje (+1 akapit)	jedyny opis pipeline’u i jego pułapek; nie dubluje README. Brakuje zdania, co zrobić z wynikiem: <code>data_nowe.json</code> nie trafia do <code>public/data.json</code> (źródłem prawdy jest baza — wynik trzeba przenieść migracją SQL jak <code>migrate-2026-09-20-bdot10k.sql</code>), a <code>kandydaci_panel.json</code> kopiuje się jako <code>public/kandydaci-zbiorniki.json</code> (nazwa z <code>config.json</code> → <code>snapshot.candidatesFile</code>)

Plik	Dla kogo	Aktualny wobec kodu?	Decyzja	Uzasadnienie
tools/qa/README.md	b, d	tak — podkomendy pobierz/ocr/parsuj/porownaj/all , opcje - -config/- -wyjscie/-v , kody wyjścia 0/1/2, sekcje konfiguracji, wymagania (requirements.txt) zgodne z __main__.py i config.json	zostaje	związły, techniczny, potrzebny każdemu, kto będzie sprawdzał dane po nowym wykazie
docs/raport- zbiorniki.md	b (historycznie: Kamil)	nie — opisuje stan sprzed migracji (39 zbiorników a:1 , dziś 27; 10 „pewnych” już naniesiono), zawiera narrację („Szczерze o jakości”, „progi dobrano po obejrzeniu...”)	usunąć z repo (→ _prywatne/), sekcję 7 „Przypadki podejrzane” przenieść do Issues	to raport z jednorazowej analizy; trwale informacje (metoda, progi) są już w tools/bdot/README.md \$Algorytmy i w public/kandydaci- zbiorniki.json → meta.progi ; README linkuje go jako „omówienie”, po usunięciu link znika
docs/rzeki-raport.json	nikt	plik pośredni pipeline'u (work/rzeki_raport.json , krok 5), nigdzie nielinkowany, odtwarzalny komendą zszuj ; zawiera 25 rzek (dziś 28)	usunąć z repo	artefakt roboczy, nie dokumentacja
docs/specs/2026-06-11- github-pages-open- source-design.md	nikt (notatka projektowa)	nie — Leaflet z CDN, data.json w katalogu głównym, „bez CI”, .gitignore wykluczający wykaz-wod-ocr.txt , odwołanie do PROMPT.md	usunąć z repo	opisuje architekturę, której nie ma; myli czytelnika b/d
docs/specs/2026-06-24- panel-operatora- supabase-design.md	nikt	nie — config.js z SUPABASE_URL , db/seed.mjs (nie istnieje), RLS „dla roli authenticated” (dziś is_operator()), CDN, „bez kroku budowania”	usunąć z repo	jw.; dodatkowo „Decyzje (zatwierdzone przez użytkownika)” to ślad wewnętrznego procesu
docs/specs/2026-06-25- edycja-rzek-granic- design.md	nikt	nie — db/seed-rivers-granice.sql (nie istnieje), Geoman z CDN	usunąć z repo	jw.
docs/specs/2026-09-20- polskie-zrodla-danych- design.md	nikt	częściowo — ścieżki data/kandydaci- zbiorniki.json i basemaps.js w katalogu głównym są błędne (dziś public/... , src/...), „geo: zamiast Google Maps” (dziś geo: tylko Android, domyślnie Apple Maps), 25 rzek (dziś 28), „etap 2”	usunąć z repo	wartościowe fakty (liczby, pułapki) są już w tools/bdot/README.md ; reszta to dziennik prac
db/schema.sql (nagłówek i komentarze)	d	tak — tabele, trigger, operators , is_operator() , RLS, revoke/grant zgodne z README	zostaje	komentarze przy kolumnach są jedyną „dokumentacją modelu” — wystarczają
db/migrate-2026-09-20- bdot10k.sql (nagłówek)	d	tak, ale blok kontrolny wymaga dokładnie 25 rzek — po migracji krainy pstrąga (28 rzek) ponowne uruchomienie kończy się wyjątkiem, więc „bezpieczna do wielokrotnego uruchomienia” przestaje być prawdą	zostaje do czasu uruchomienia; potem usunąć	patrz reguła w §4
db/migrate-2026-09-21- uprawnienia.sql	d	tak; treść jest identyczna z fragmentem schema.sql (te same revoke/grant) — potrzebna wyłącznie istniejącej bazie	zostaje do czasu uruchomienia; potem usunąć	jw.
db/migrate-2026-09-21- kraina-pstraga.sql	d	tak (3 insert ... where not exists , 2 update , kontrola 18 odcinków gor)	zostaje do czasu uruchomienia; potem usunąć	jw.
.github/workflows/*.yaml (nagłówki)	d	tak — opisy zgodne z krokami; crony 03:15 i 7 */6 zgodne z README	zostają	nagłówki 2–4 linie, wystarczające
.github/dependabot.yaml	d	tak	zostaje	—
tools/snapshot/*.mjs (docstringi)	d	tak — PZW_OUT , PZW_FORCE , kod wyjścia 2, argument ścieżki w validate / seed-sql zgodne z kodem	zostają	—

Plik	Dla kogo	Aktualny wobec kodu?	Decyzja	Uzasadnienie
config.json	b, d	jest kodem; wszystkie sekcje z README istnieją (site , supabase , snapshot , map , basemaps , links) i wszystkie są czytane (src/config.js , vite.config.js , workflowy przez jq)	zostaje	links.report.email pusty → zgłoszenia idą do Issues (zgodnie z main.js → reportUrl) — README to teraz mówi wprost
package.json	b, d	tak — skrypty dev/build/preview/export/validate/seed-sql/check istnieją i są cytowane poprawnie	zostaje	—
LICENSE	wszyscy	MIT, © Kamil Kornacki	zostaje	MIT pozwala Okręgowi na dowolne użycie i modyfikacje bez zmiany pliku; licencja danych opisana w README (§Licencja i źródła) — wystarczy
public/kandydaci-zbiorniki.json → meta	d	tak (27 wyników, progi)	zostaje	to dane, nie dokumentacja; meta.uwaga zawiera datę i ślad decyzji operatora — akceptowalne

3. Nieścisłości README (obecnego) względem kodu

Potwierdzone przez porównanie z kodem; wszystkie poprawione w załączonym README.md.

- Stan migracji nie zgadza się z danymi.** README: migrate-2026-09-21-kraina-pstraga.sql „do uruchomienia”, a public/data.json już zawiera 3 nowe cieki (28 rzek, 18 gor, 7 212 wierzchołków; meta.snapshot 2026-09-20T20:24Z, czyli plik zbudowano ręcznie). Dopóki migracja nie zostanie uruchomiona: mapa na żywo pokazuje 25 rzek (baza nadpisuje snapshot warstwa po warstwie, src/data.js → loadData), a nocny snapshot.yml **kończy się błędem co noc** (bezpiecznik w export.mjs: baza 25 < snapshot 28). Kolumna „Stan” w README z natury się starzeje → usunięta; zamiast niej reguła: migracja to plik jednorazowy, usuwany po uruchomieniu na produkcji.
- „Skrypty są idempotentne”** — pojedynczo tak, ale migrate-2026-09-20-bdot10k.sql po migracji krainy pstrąga rzuca wyjątek (Nieoczekiwana liczba rzek: 28). Kolejność i jednorazowość trzeba zaznaczyć (zrobione).
- npm run check „to samo robi CI” — CI dodatkowo robi npm run build. Dorecyzowane.
- Własna domena: „ustaw site.basePath”** — trzeba też zmienić site.url, bo używa go healthcheck.yml (jq -r .site.url). Dopisane.
- Snapshot „strona używa go, gdy baza nie odpowiada”** — nieprecyzyjne: loadData() zawsze wczytuje snapshot, potem nadpisuje każdą kolekcję z bazy, jeśli odczyt się powiódł. Skutek praktyczny ten sam, ale opis poprawiony, bo od tego zależy zrozumienie punktu 1.
- Struktura snapshotu pomija meta** (zrodla, uwaga, snapshot), a main.js i wykaz.js czytają meta.snapshot do napisu „Stan danych”. Dodane.
- tools/bdot/README.md** twierdzi, że wyjściem jest „nowy data.json”, a w repo źródłem prawdy jest baza — wynik pipeline’u trafia do niej przez migrację SQL, a plik kandydatów zmienia nazwę na public/kandydaci-zbiorniki.json. Nie jest to błąd README głównego, ale luka, którą osoba d odczuje przy następnej aktualizacji BDOT10k. Proponowany akapit do tools/bdot/README.md (sekcja „Pliki wejściowe i wyjściowe”, na końcu):

Źródłem prawdy dla mapy jest baza, nie public/data.json. Nowe geometrie i pinezki klasy pewne z data_nowe.json przenosi się do bazy skryptem SQL w db/ (wzór: migrate-2026-09-20-bdot10k.sql), a kandydaci_panel.json kopiuje jako public/kandydaci-zbiorniki.json (nazwa z config.json → snapshot.candidatesFile). Snapshot odświeży się sam następnej nocy.

- Podkłady: „mapa ogólnogeograficzna”** — warstwa w config.json nazywa się „Mapa” (G2_MOBILE_500); w README nazwane „mapa ogólna”, żeby pasowało do przełącznika.
- „Nawiguj otwiera domyślną nawigację telefonu”** — w kodzie: geo: tylko dla Androida, w pozostałych przypadkach link do Apple Maps (links.navigation.default). Skrócone do „aplikacja nawigacyjna telefonu”.
- Region Frankfurt, liczba „27 zbiorników”, nazwy konkretnych cieków w „Znanych brakach”** — poprawne dziś, ale to stan danych, który operator zmienia w panelu bez commitów; usunięte z README, miejsce na to jest w Issues.
- Link do docs/raport-zbiorniki.md** — raport opisuje 39 zbiorników (stan sprzed migracji); po przeniesieniu pliku poza repo link usunięty.

5. **links.report.email** pusty — README mówiło tylko „report.email Okręgu”; kod (`main.js` → `reportUrl`) przy pustym adresie kieruje do formularza Issues. Dopisane, bo Okręg musi wiedzieć, gdzie trafiają zgłoszenia.

Rzeczy sprawdzone i **zgodne**: `npm run seed-sql` istnieje (`tools/snapshot/seed-sql.mjs`); `db/schema.sql` zawiera tabele, allow-listę `operators`, `RLS`, `is_operator()` i `revoke/grant`; tabela workflowów odpowiada plikom i cronom; wszystkie 6 sekcji `config.json` z README istnieje; opis panelu (Geoman, kandydaci z powierzchnią/odległością/oceną, przeniesienie pinezki po kliknięciu) odpowiada `src/admin.js`; „W czasie działania strona łączy się wyłącznie z Geoportalem i własną bazą” potwierdza CSP w `vite.config.js`; liczby w README (27 a:1) zgadzają się z `public/data.json` i z `kandydaci-zbiorniki.json` (27 wyników).

Poza dokumentacją, ale wykryte przy okazji (do osobnej decyzji)

- **Nocny snapshot nie publikuje się na Pages.** `snapshot.yml` commituje przez `GITHUB_TOKEN`, a push wykonany tym tokenem **nie uruchamia** `deploy.yml` (ograniczenie GitHub Actions). Repo ma świeży `public/data.json`, ale strona serwuje snapshot z ostatniego builda po ludzkim pushu. Mapa i tak czyta bazę na żywo, więc użytkownik tego nie widzi — ale kopia awaryjna na stronie się starzeje. Najprostsza poprawka: w `snapshot.yml` po pushu `gh workflow run deploy.yml` (uprawnienie `actions: write`; `workflow_dispatch` jest wyjątkiem od tego ograniczenia).
- Przy pustej sekcji `supabase` w `config.json` `healthcheck.yml` będzie co 6 h zakładał issue o awarii bazy (curl na pusty adres). README mówi, że wtedy „mapa działa na snapshotcie” — prawda, ale trzeba by wyłączyć `healthcheck`. Nieistotne, dopóki baza istnieje.

4. Docelowa, minimalna struktura dokumentacji

<code>README.md</code>	← jedyny punkt wejścia (a, b, d); ~130 linii
<code>docs/instrukcja-operatora.md</code>	← jedyny dokument nietechniczny (c); linkowany z README i (docelowo) z <code>admin.html</code>
<code>tools/bdot/README.md</code>	← bez zmian + 1 akapit „co z wynikiem”
<code>tools/qa/README.md</code>	← bez zmian
<code>db/schema.sql</code>	← komentarze przy kolumnach = dokumentacja modelu
<code>db/migrate-*.sql</code>	← tylko NIEuruchomione; po uruchomieniu na produkcji <code>`git rm`</code>
<code>.github/workflows/*.yml</code>	← 2–4-liniowe nagłówki, bez zmian

Kolejność sekcji README (i cel długości): tytuł + 6 punktów „co jest na mapie” → Zastrzeżenie → Jak to działa (tabela 5 wierszy) → Uruchomienie lokalne → Struktura repozytorium (+ format snapshotu) → Konfiguracja → Baza i panel (5 kroków + reguła migracji + link do instrukcji) → Automatyzacje → Jak pomóc → Licencja i źródła.

Reguła dla migracji (zamiast kolumny „Stan”): migracja jest jednorazowa; na świeżej bazie wystarczy `schema.sql` + `seed`; po uruchomieniu na bazie produkcyjnej plik usuwa się z repo. Wtedy stan repo mówi sam za siebie: „jeśli w `db/` jest `migrate-*.sql`, trzeba go uruchomić”. Nie wymaga żadnej dodatkowej dokumentacji ani tabeli.

Instrukcja operatora — forma i miejsce

- **Forma: jeden plik Markdown w repo (`docs/instrukcja-operatora.md`).** Powody: wersjonowany razem z panelem (zmiana etykiety przycisku = zmiana w tym samym PR), GitHub renderuje go czytelnie pod stałym adresem, Okręg może go wydrukować do PDF z przeglądarki, gdy zechce (PDF poza repo starzeje się bez śladu). Strona w aplikacji (`instrukcja.html`) to więcej kodu do utrzymania — niepotrzebne, bo wystarczy link z karty logowania w `admin.html` do wersji na GitHubie.
- Treść: tylko to, co jest w `src/admin.js` i `admin.html` (etykiety przycisków, komunikaty z `tlumaczBład`, zachowanie kandydatów, Geoman). Bez nazwisk i adresów — „osoba techniczna wskazana przez Okręg”.
- Sugerowana zmiana w kodzie (jedna linia w `admin.html`, w karcie logowania pod formularzem): link „Instrukcja obsługi panelu” do `https://github.com/korntech/mapa-wod-pzw-rzeszow/blob/main/docs/instrukcja-operatora.md`.

5. Wiki GitHub — rekomendacja: NIE

Argument	Ocena w tym projekcie
Wersjonowanie	Wiki to osobne repozytorium git (<code>*.wiki.git</code>), poza PR-ami, CI i historią kodu. Zmiana w <code>admin.js</code> i w opisie panelu nie mogą trafić w jednym commicie → drift gwarantowany.
Drift względem kodu	Cała obecna dokumentacja mieści się w ~500 liniach i jest w 90 % listą komend/ścieżek — to dokładnie rodzaj treści, który w wiki najszybciej się dezaktualizuje, a w README obok kodu jest widoczny w diffie.

Argument	Ocena w tym projekcie
Osoba nietechniczna	Operator nie szuka wiki na GitHubie; potrzebuje jednego linku z panelu do jednej strony. Plik w repo daje to samo.
„Generowanie z kodu”	Nie ma z czego generować: brak API, brak JSDoc-a w stylu bibliotecznym, moduły to 7 plików po ~100 linii. Generator wyprodukuję listę funkcji (<code>initBasemaps</code> , <code>loadData</code> ...), której żaden z czterech czytelników nie potrzebuje. Jedyne „generowalne” treści (opcje CLI narzędzi Pythona) są już opisane w README narzędzi i w <code>--help</code> .
Przekazanie Okręgowi	Przy przeniesieniu repo do organizacji Okręgu wiki trzeba migrować osobno; wiki nie działa w forkach; uprawnienia do wiki są osobne. Kolejna rzecz do pilnowania w organizacji bez IT.
Zalety wiki	Edycja w przeglądarce bez PR — ale to samo daje edycja <code>.md</code> w interfejsie GitHuba (przycisk „Edit”), z tą różnicą, że przechodzi przez PR/CI.

Wniosek: dokumentacja zostaje w repo jako 4 pliki Markdown (README + 2 README narzędzi + instrukcja operatora). Jeśli kiedyś urośnie ponad to, sygnałem będzie długość README, nie brak wiki.

6. Komendy do wykonania (w klonie repo, gałąź robocza)

```
# 1. dokumenty wewnętrzne poza repo (_prywatne/ jest w .gitignore; uwaga: `git mv` do katalogu
# ignorowanego NIE usuwa plików z indeksu – śledzone pliki pozostają śledzone, dlatego mv + git rm)
mkdir -p _prywatne/docs
mv docs/specs docs/raport-zbiorniki.md docs/rzeki-raport.json _prywatne/docs/
git rm -r docs/specs docs/raport-zbiorniki.md docs/rzeki-raport.json

# 2. nowa dokumentacja
cp README.md README.md
cp docs/instrukcja-operatora.md docs/instrukcja-operatora.md
git add README.md docs/instrukcja-operatora.md

# 3. akapit „co z wynikiem” w tools/bdot/README.md (treść w §3 pkt 7) – ręcznie
# 4. (opcjonalnie, 1 linia) link do instrukcji w admin.html, karta logowania

# 5. po uruchomieniu migracji na bazie produkcyjnej – w tej kolejności:
# migrate-2026-09-20-bdot10k.sql (jeśli jeszcze nie), migrate-2026-09-21-uprawnienia.sql,
# migrate-2026-09-21-kraina-pstraga.sql
git rm db/migrate-2026-09-20-bdot10k.sql
git rm db/migrate-2026-09-21-uprawnienia.sql
git rm db/migrate-2026-09-21-kraina-pstraga.sql

git commit -m "Dokumentacja: skrócony README, instrukcja operatora, notatki projektowe poza repo"
```

Issues do założenia (zamiast usuwanych treści z README i raportu):

1. „Zbiorniki z lokalizacją przybliżoną (27) — do weryfikacji w panelu” — z listą nazw z `public/kandydaci-zbiorniki.json` i przypadkami z §7 raportu (Mrowla–Lipie ×5, Rzemień 1-2 vs 3, Błażkowa, Strzegocice, Otałęż, Zwiężczyca, Czarna Sędziszowska, Blizne).
2. „Trzebońnica — odcinek źródłowy bez geometrii w BDOT10k (ok. 1,2 km)”.
3. „Granica obwodów Wiślok 3/4 (most kolejowy w Rzeszowie) — przybliżona”.
4. „Nazewnictwo: Stobnica (wykaz) / Stopnica (BDOT10k)”.
5. „Pomysły: tryb offline (PWA), eksport GPX, zdjęcia łowisk”.
6. „snapshot.yml nie uruchamia deploy.yml (GITHUB_TOKEN)” — patrz §3, sekcja „Poza dokumentacją”.